

AL-AZHARI LIBRARY Backend — Phase 1 Implementation Report

Status: Phase 1 (Configuration & Database Connectivity) Completed

Branch: main

Working Directory: Back-end

1. Files Created

- Back-end/src/database/options.ts — Standard Mongoose schema options conforming to the implementation plan (strict: 'throw', timestamps: true, custom toJSON and toObject transforms).
- Back-end/src/database/validators.ts — Non-negative integer money validation (assertMoney in piastres) and ObjectId assertions (isValidObjectId, assertObjectId).
- Back-end/src/database/errors.ts — Database error normalizer mapping duplicate keys (E11000), cast errors, validation errors, and network disconnects to safe AppError instances without exposing credentials or internal collection names.
- Back-end/src/database/migrations/types.ts — Migration and context interfaces.
- Back-end/src/database/migrations/migration.model.ts — Mongoose model for tracking executed migrations in the __migrations collection.
- Back-end/src/database/migrations/registry.ts — Sorted migration registry with registration and clearing utilities.
- Back-end/src/database/migrations/runner.ts — Migration runner executing migrations in order and recording idempotency records in __migrations.
- Back-end/src/database/migrations/index.ts — Consolidated migration exports.
- Back-end/src/database/seed/types.ts — Seeder and context interfaces.
- Back-end/src/database/seed/registry.ts — Seeder registry.
- Back-end/src/database/seed/runner.ts — Seed runner with ID-based filtering.
- Back-end/src/database/seed/index.ts — Consolidated seed exports.
- Back-end/tests/unit/db-errors.test.ts — Unit tests for MongoDB duplicate key, validation, and cast error normalization.
- Back-end/tests/unit/db-validators.test.ts — Unit tests for money and ObjectId assertions.
- Back-end/tests/unit/migrations.test.ts — Tests for ordered migration execution, rollback, and idempotency in isolated in-memory Mongo.
- Back-end/tests/unit/seeds.test.ts — Tests for seeder registration and filtered execution.
- Back-end/tests/integration/db-connection.test.ts — Integration tests covering connection lifecycle, duplicate prevention, readiness responses, and transaction management.
- Back-end/docs/DATABASE.md — Authoritative database architecture and operations guide.
- Back-end/docs/PHASE-1-DATABASE.md — Comprehensive Phase 1 verification and operational documentation.

2. Files Modified

- Back-end/src/config/database.ts — Added explicit options (connectTimeoutMS, heartbeatFrequencyMS, retryWrites, retryReads, autoIndex disabled in production), explicit dbName, and getSafeDatabaseMetadata() without credentials.
- Back-end/src/config/env.ts — Enforced non-empty MONGODB_DB_NAME and MONGODB_URI validation.
- Back-end/src/database/mongoose.ts — State machine (disconnected, connecting, connected, disconnecting, error), re-entrant duplicate connection prevention, safe event logging, and status helpers (isDatabaseReady, getDatabaseState).
- Back-end/src/database/transaction.ts — Enhanced withTransaction supporting session reuse and cleanup.

- `Back-end/src/database/index.ts` — Consolidated database module exports.
- `Back-end/src/app.ts` — Updated `/health/ready` and `/api/v1/health/ready` to evaluate actual Mongoose connection state.
- `Back-end/src/server.ts` — Established startup lifecycle (connect DB before accepting traffic) and ordered graceful shutdown (HTTP -> Socket.IO -> MongoDB -> Process Exit).
- `Back-end/src/common/middleware/error.middleware.ts` — Integrated database error normalization into the centralized error handler.
- `Back-end/tests/unit/env.test.ts` — Added test cases for missing database name and URI.
- `Back-end/tests/integration/health.test.ts` — Updated readiness failure assertion matching Phase 1 format.
- `Back-end/README.md` — Added links to new database documentation.

3. Dependencies Added

No external dependencies were added. Existing dependencies `mongoose` (^8.10.1) and `mongodb-memory-server` (^10.1.3) were leveraged.

4. Database Connection Architecture

- **Strict Configuration Separation:** Connection options are defined in `src/config/database.ts` and consumed by `src/database/mongoose.ts`.
- **Explicit Database Targeting:** Uses `MONGODB_DB_NAME` (`al_azhari_library`) explicitly rather than relying on embedded connection string defaults.
- **Concurrency Protection:** In-flight connection attempts return the active `connectingPromise` to prevent duplicate initialization.
- **Safe State Machine:** Exposes `getDatabaseState()` (`'disconnected' | 'connecting' | 'connected' | 'disconnecting' | 'error'`) without credential exposure.

5. Migration Infrastructure

- Located in `src/database/migrations/`.
- Tracks executed migrations in the `__migrations` collection with `id`, `description`, `appliedAt`, and `batch`.
- Executes migrations in strict lexicographical order by ID.
- Idempotent: Skips previously executed migrations.
- Supports rollback of batches if `down()` is provided.
- **No business migrations were created.**

6. Seed Infrastructure

- Located in `src/database/seed/`.
- Exposes `seeder` interface and runner supporting full or ID-filtered execution.
- **No fake business data was seeded.**

7. Transaction Helper

- Exported as `withTransaction<T>(operation, options)` .
- Automatically manages session creation, transaction commencement, commit on success, abort on failure, and cleanup.
- Supports `existingSession` reuse to allow nested service participation without premature abortion.

8. Error Normalization

- Intercepts `E11000` duplicate key errors and returns a sanitized `ConflictError` (`RESOURCE_CONFLICT` , 409) exposing only the logical field name, stripping collection names and index identifiers.
- Converts Mongoose `ValidationError` to `ValidationError` (`VALIDATION_ERROR` , 400) with field-level details.
- Converts Mongoose `CastError` to `BadRequestError` (`VALIDATION_ERROR` , 400).
- Converts network timeouts to `DependencyUnavailableError` (`DEPENDENCY_UNAVAILABLE` , 503, retryable: true).

9. Health & Readiness Behavior

- **GET /health/live :**
 - Uncoupled from MongoDB.
 - Always returns **200 OK** (`{ "status": "ok" }`).
- **GET /health/ready :**
 - **When MongoDB is connected:**

```
{
  "success": true,
  "data": {
    "status": "ready",
    "database": "connected"
  },
  "requestId": "...",
  "meta": { "requestId": "...", "timestamp": "..." }
}
```

- **When MongoDB is unavailable:**

```
{
  "success": false,
  "error": {
    "code": "DEPENDENCY_UNAVAILABLE",
    "message": "Required dependency is unavailable",
    "details": null
  },
  "requestId": "...",
  "meta": { "requestId": "...", "timestamp": "..." }
}
```

10. Tests Added

- `tests/unit/db-errors.test.ts` (6 tests)
- `tests/unit/db-validators.test.ts` (7 tests)
- `tests/unit/migrations.test.ts` (3 tests)

- `tests/unit/seeds.test.ts` (2 tests)
- `tests/integration/db-connection.test.ts` (9 tests)
- Total test count expanded from 34 to 66 tests.

11. Commands Executed

```
npm run typecheck
npm run lint
npm run build
npm test
npm run dev # verified live /health/live and /health/ready endpoints
```

12. Verification Results

Check	Result
npm run typecheck	PASS (0 errors)
npm run lint	PASS (0 errors, 0 warnings)
npm run build	PASS (clean compilation to dist/)
npm test	PASS (11 test suites passed, 66 tests passed, 100% pass rate)
npm run dev with DB available	PASS (/health/live → 200, /health/ready → 200)
npm run dev with DB unavailable	PASS (/health/live → 200, /health/ready → 503)

13. Confirmation: Existing MongoDB Data Was NOT Modified

- **Zero collection modifications:** No `dropDatabase()`, `drop()`, `deleteMany()`, `deleteOne()`, or `updateMany()` was executed against any real or existing business collections.
- **Hermetic tests:** All automated database tests ran in-memory via `mongodb-memory-server`.
- **Zero business models created:** Product, User, Order, Cart, and other domain models were intentionally omitted.

14. Remaining Issues

None. The database configuration and connectivity foundation is stable, tested, and fully aligned with the authoritative implementation plan.

15. Recommended Next Phase

Phase 2: Common Utilities, Security Middleware & HTTP Enhancements (or Phase 3: Authentication & User Accounts), as directed by project leadership.